Step 5.5 – Persistent journal (ticket #11) · final

Project 2 – Acme Analytics Linux platform · Phase 5 · Day 18, 13 Nov 2026 · app01 and db01 ✔ · stor01: same file, to do

Phase 5 · Step 5.5 – Persistent journal (each server keeps its own logs safely)

Project 2 – Acme Analytics Linux platform · Day 18 · ticket #11

GLPI · Ticket #11 · New

Title	**After the Sunday update restart – are db01's logs from before the restart still there?**
Requester	Sara (IT lead)
Category	Linux › Logging
Priority	Medium
Servers	app01, db01, stor01

"On Sunday night all servers restarted for updates. On Monday someone asked what db01 did **just before** the restart. stor01 has a copy (Step 5.4) – but only of lines that were sent in time, and only as text. I want every server to **keep its own full journal across restarts**, and I don't want the journal to **fill the disk**. Please check it, set it, and prove it with a restart."

Volatile vs persistent journal – the picture

Volatile (kept in RAM)
`/run/log/journal/`
restart → **all logs gone**
`journalctl --list-boots` shows only **0**
`journalctl -b -1` → "no persistent journal"

Persistent (saved on disk)
`/var/log/journal/<machine-id>/`
restart → **logs stay**
`--list-boots` shows 0, -1, -2 …
`journalctl -b -1` → the run before the restart

One setting decides it – a drop-in file `/etc/systemd/journald.conf.d/10-persistent.conf` : `Storage=persistent` keep on disk · `SystemMaxUse=500M` never bigger than 500 MB · `SystemKeepFree=1G` always leave 1 GB free · `MaxRetentionSec=1month` delete older than 1 month
Two places then: **own journal** on each server (5.5, binary, full) + **copy on stor01** (5.4, text, sorted).

What earlier steps give this one

Ticket #9: we read Saturday's kernel log with `journalctl -k -b -3` – that only works if old runs are kept
Step 5.2: `logger -t` to write a test line
Step 5.4: stor01 has a text copy – this step protects the **own** copy on each server
Phase 4: drop-in files and limits (archive worker)

New words

journald – the service that keeps all logs of a server
volatile – kept in RAM, lost at restart · **persistent** – saved on disk
boot – one run from start to restart (0 = now, -1 = before)
drop-in – a small extra settings file, the main file stays untouched

Plan

1. **A · Check** on app01: where is the journal now, how many boots, how big?
2. **B · Set it:** drop-in file with Storage=persistent + size limits, restart journald
3. **C · Prove it:** logger line → restart app01 → find the line with `-b -1`
4. **D · Same file on db01 and stor01**
5. **E · Close ticket #11**

Final paragraph – logs on a server: who keeps them, who cleans them

1 · journald keeps everything. journald writes **every** line of the server – from the **first second of boot to the last second of shutdown** – into `/var/log/journal/` (read with `journalctl`). It is **persistent** only when saved **on the disk**; in **RAM** (`/run/log/journal/`) every restart wipes it. Like `sync` (Day 15), journald writes its lines to the disk in small batches – every few minutes, at once for serious errors – so a restart loses almost nothing. Then `journalctl -b -1` shows the whole run before a restart.

2 · rsyslog keeps a subset. rsyslog runs only in the **middle** of each run – it starts later than journald and stops earlier. While running it takes each line from journald, writes a **text copy on the same server** (`/var/log/syslog`, `/var/log/auth.log`) and **sends** it live over TCP 514 to **stor01** (`/var/log/remote/<server>/<program>.log`). So stor01 has the middle of each run, often **not the start and the end**.

3 · Programs with their own files – nginx (`/var/log/nginx/`), PostgreSQL (`/var/log/postgresql/`) – reach journald and stor01 only with one setting (Step 5.4b).

4 · Who cleans what, so no disk ever fills:

What	Where	Cleaned by	Rules
journald's files	`/var/log/journal/` (every server)	**journald itself**	**Step 5.5:** at most 500 MB · always leave 1 GB of the disk free · delete after 1 month – the **oldest lines** go first, the **latest** stay
text files on each server	`/var/log/syslog`, `auth.log`, `/var/log/nginx/*.log` …	**logrotate**	built into Ubuntu, runs daily; rules ready-made in `/etc/logrotate.d/` : a fresh file each day/week, old ones become **versions** (`.1`, `.2.gz` …), after N versions the oldest is deleted
stor01's central files	`/var/log/remote/…`	**logrotate**	**Step 5.6:** our own rule – Ubuntu does not know this folder

The journald rules clean only journald's files; logrotate follows only its own rules. Deleted journald lines are gone **from that server** – stor01 often still has them, because it keeps its copy longer.

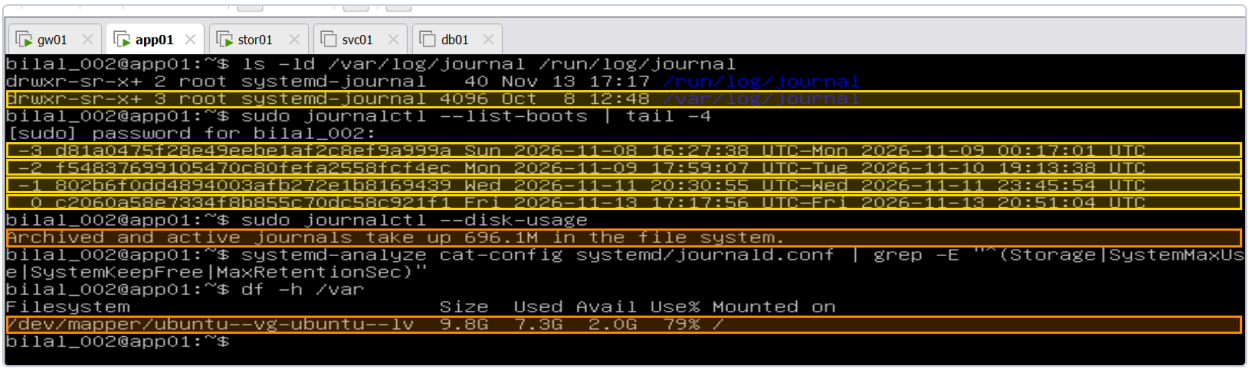
5 · Result of Step 5.5: every server's log lives in **two places** – its **own journal** (complete, the recent weeks) and **stor01** (the middle of each run, kept longer).

Part A · Check – where does app01 keep its journal now?

A Look before changing anything (read-only)

app01

```
ls -ld /var/log/journal /run/log/journal     # which place: disk (/var/log/journal) or RAM (/run/log/journal)?
sudo journalctl --list-boots | tail -4       # how many earlier runs are still kept?
sudo journalctl --disk-usage                 # how much space does the journal use?
systemd-analyze cat-config systemd/journald.conf | grep -E
"^(Storage|SystemMaxUse|SystemKeepFree|MaxRetentionSec)"   # any rules written down?
df -h /var                                   # how big and how full is the disk?
```



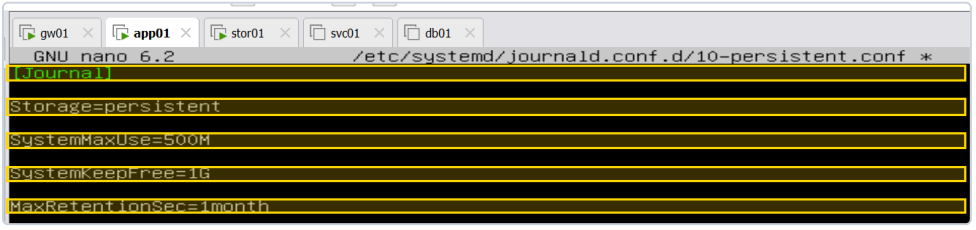
Command	What we saw	Meaning
ls -ld	both exist: `/var/log/journal` (disk, since 8 Oct) and `/run/log/journal` (RAM)	already on the disk; the RAM folder is only used for the first seconds of boot
--list-boots	-3 (8 Nov) · -2 (9 Nov) · -1 (11 Nov) · 0 (now)	old runs survived restarts
--disk-usage	**696.1M** (orange)	bigger than 500 MB – no limit was set
cat-config | grep	no lines	none of our rules written down – only Ubuntu's defaults
df -h /var	9.8G · **79 % used** · **2.0G free** (orange)	close to the 85 % warning – a limit is needed

Part B · Set it – one drop-in file

B1 Write the rules

app01

```
sudo mkdir -p /etc/systemd/journald.conf.d                  # journald's drop-in folder
sudo nano /etc/systemd/journald.conf.d/10-persistent.conf   # save Ctrl+O, Enter · exit Ctrl+X
```

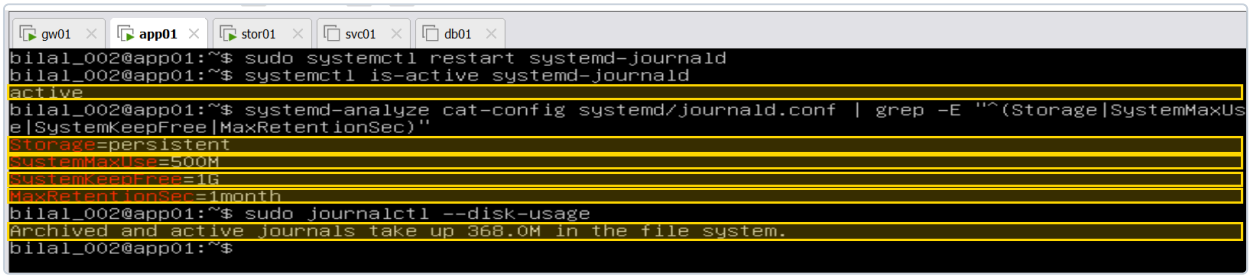


Line	Meaning
`[Journal]`	section name – journald reads only settings under it
`Storage=persistent`	always keep the journal on the disk (`/var/log/journal`)
`SystemMaxUse=500M`	the journal may take up at most 500 MB of the disk – then the oldest lines are deleted
`SystemKeepFree=1G`	at least 1 GB of the disk must always stay free – journald deletes its own old lines earlier if needed
`MaxRetentionSec=1month`	lines older than 1 month are deleted

B2 Load and check

app01

```
sudo systemctl restart systemd-journald       # journald reads the new file
systemctl is-active systemd-journald          # still running?
systemd-analyze cat-config systemd/journald.conf | grep -E
"^(Storage|SystemMaxUse|SystemKeepFree|MaxRetentionSec)"   # rules in force?
sudo journalctl --disk-usage                  # size now
```



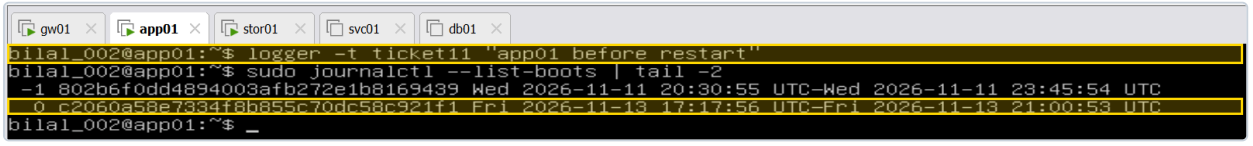
What we saw: `active` · all **4 rules** in force · the journal went from **696.1M to 368.0M** – journald deleted its oldest files at once to get under 500 MB (the oldest runs, 8 and 9 Nov, are gone – expected).

Part C · Prove it with a restart

C1 A line before the restart

app01

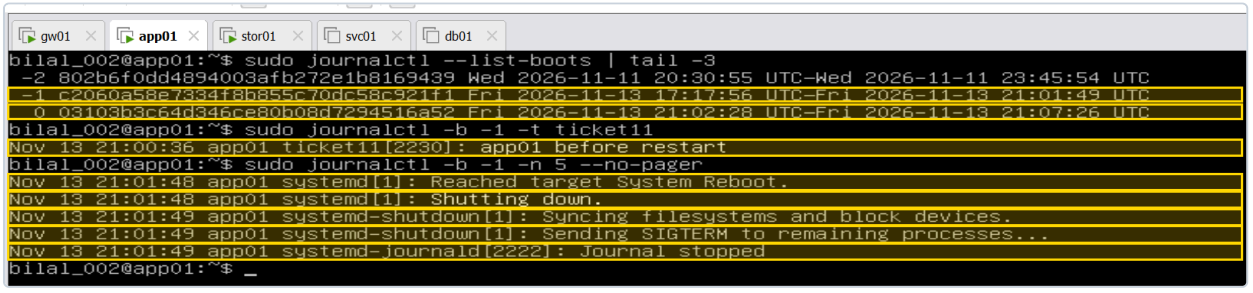
```
logger -t ticket11 "app01 before restart"     # a test line written BEFORE the restart
sudo journalctl --list-boots | tail -2        # this run is 0 now
sudo reboot                                   # restart app01; log in again from gw01 after ~30 s
```

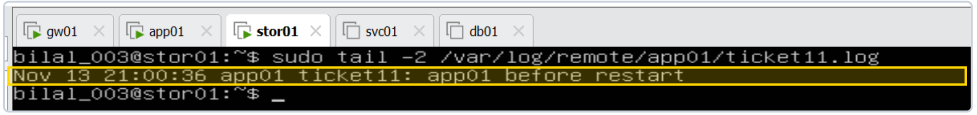


C2 After the restart – did it survive?

app01 · stor01

```
# ── on app01 ──
sudo journalctl --list-boots | tail -3        # a new run 0 – the run before is now -1
sudo journalctl -b -1 -t ticket11             # our line from the run before the restart
sudo journalctl -b -1 -n 5 --no-pager         # the last 5 lines before the restart
# ── on stor01 ──
sudo tail -2 /var/log/remote/app01/ticket11.log   # the same line in the central copy
```





Command	What we saw	Meaning
--list-boots	new 0 (21:02) · -1 = 17:17 → 21:01:49	the run before the restart is kept
-b -1 -t ticket11	`app01 ticket11: app01 before restart`	**the line survived the restart** – persistent
-b -1 -n 5	Reached target System Reboot · Shutting down · Syncing filesystems · Sending SIGTERM · **Journal stopped**	the shutdown lines – stor01 usually does not get these; only the own journal has them
stor01 tail	the same test line	written while rsyslog ran → in **both places**

Two tests – and why db01 needed no reboot

	Test 1 · restart of journald (Parts B and D)	Test 2 · reboot of the whole server (Part C)
Command	`sudo systemctl restart systemd-journald`	`sudo reboot`
What restarts	only the log service, for a second – shop and database keep running	the whole server – for about a minute nothing works
New boot?	no – same run	yes – the old run becomes -1, a new 0 starts
What it checks	did the **rules** apply, does journald **clean up**? → 4 rules in force, 696M → 368M	do lines on the disk **survive into the next run**? → test line found with `-b -1`

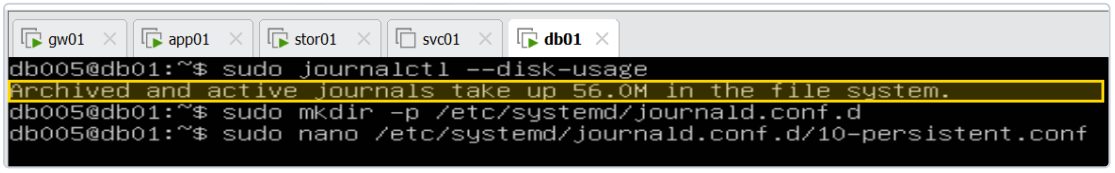
To **load** new settings both work – the difference is the cost. Rule in a company: restart **only the service** that needs the new settings. The reboot was the **proof**, done once, on app01. Part D is not a third test – it applies the same rules to db01, so its journal can never fill its disk: if db01's disk filled up, PostgreSQL could not write and the shop's database would stop.

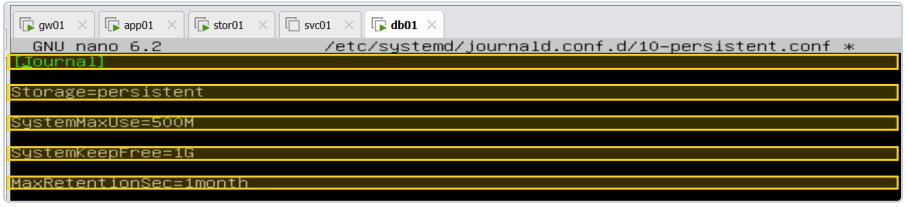
Part D · The same file on db01 (and stor01)

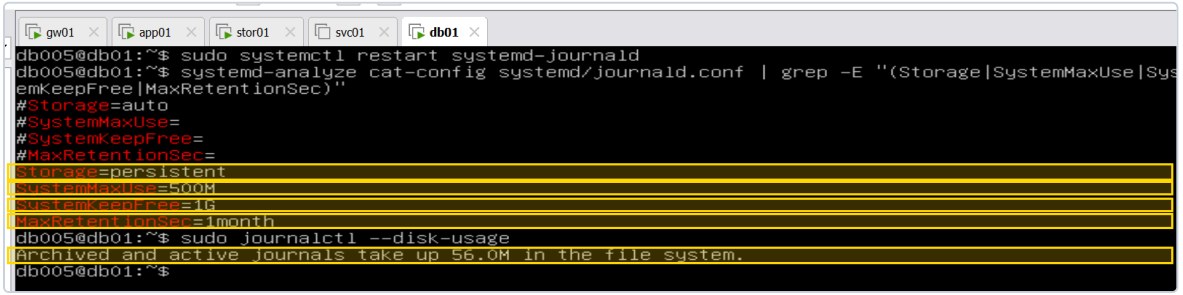
D No server restart – only journald

db01

```
sudo journalctl --disk-usage                 # size BEFORE
sudo mkdir -p /etc/systemd/journald.conf.d
sudo nano /etc/systemd/journald.conf.d/10-persistent.conf     # the same 5 lines
sudo systemctl restart systemd-journald      # load – the shop database keeps running
systemd-analyze cat-config systemd/journald.conf | grep -E "
(Storage|SystemMaxUse|SystemKeepFree|MaxRetentionSec)"
sudo journalctl --disk-usage                 # size AFTER
```







What we saw: 56.0M before and after – db01's journal was already under 500 MB, so nothing had to be deleted; the rules now guard it for the future. The grep (typed without `^`) also shows `#Storage=auto` … – Ubuntu's switched-off defaults in the main file (`#` = not active); our 4 lines below them are the ones in force. **Why no server restart:** app01 already proved the file works; db01 runs the shop database.

> **stor01 – to do:** the same 5 lines and the same commands as on db01 (user `bilal_003@stor01`).

Part E · Ticket #11 solved

GLPI · Assistance › Tickets › #11

Title	**After the Sunday update restart – are db01's logs from before the restart still there?**
Status	**Solved**
Requester	Sara (IT lead) · Category: Linux › Logging · Priority: Medium
Servers	app01, db01 (stor01: same file, to do)
Opened / solved	13 Nov 2026 · same day

✔ Solution

What was found: app01's journal was already on the disk (`/var/log/journal` , 4 earlier runs kept) – but with **no limit**: it used 696 MB on a disk that is 79 % full with 2 GB free.

What was done: one drop-in file `/etc/systemd/journald.conf.d/10-persistent.conf` on app01 and db01: `Storage=persistent` · `SystemMaxUse=500M` · `SystemKeepFree=1G` · `MaxRetentionSec=1month` . Main file untouched, only journald restarted on db01 (no server restart).

Proof:
• app01: right after loading the rules the journal shrank from 696 MB to 368 MB – journald deleted its oldest files to get under 500 MB.
• app01: a test line written before a restart is still found with `journalctl -b -1 -t ticket11` , together with the shutdown lines ("Shutting down" … "Journal stopped") – lines stor01 does not get.
• stor01 has the same test line – two places.
• db01: rules active, journal 56 MB (already under the limit).

For you: every server keeps its full log across restarts, and the journal can never fill a disk. The central files on stor01 get their own limit next (logrotate, Step 5.6).

The day in one paragraph

First we checked **where journald keeps the journal** – RAM (`/run/log/journal`) or disk (`/var/log/journal`): on the disk. The **last 4 boots** – the last 4 whole runs of the machine – proved old runs survive restarts. The journal took **696 MB** (`--disk-usage`), **no rules** were written down, and the whole disk under `/var` was **79 % full, 2 GB free** (`df -h` counts all files, not only logs). Then we made `/etc/systemd/journald.conf.d/10-persistent.conf` with the rules – on the disk; at most 500 MB, then the oldest files go; 1 GB of the disk always free; lines older than 1 month go – **restarted only journald** to load them, checked they were in force, and saw the journal shrink **696 MB → 368 MB**. Finally a **test line**, a **reboot of the whole server**, and `journalctl -b -1` still found it – the journal survives a restart. db01 got the same file with only journald restarted, so its disk is protected too.